From Trees to Stack Code

Guillermo Polito

guillermo.polito@inria.fr
@guillep













Goals

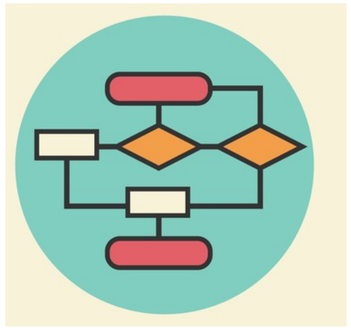
- Understanding code translation targeting stack machines

- Tree traversals and execution semantics
- Translation as tree traversal

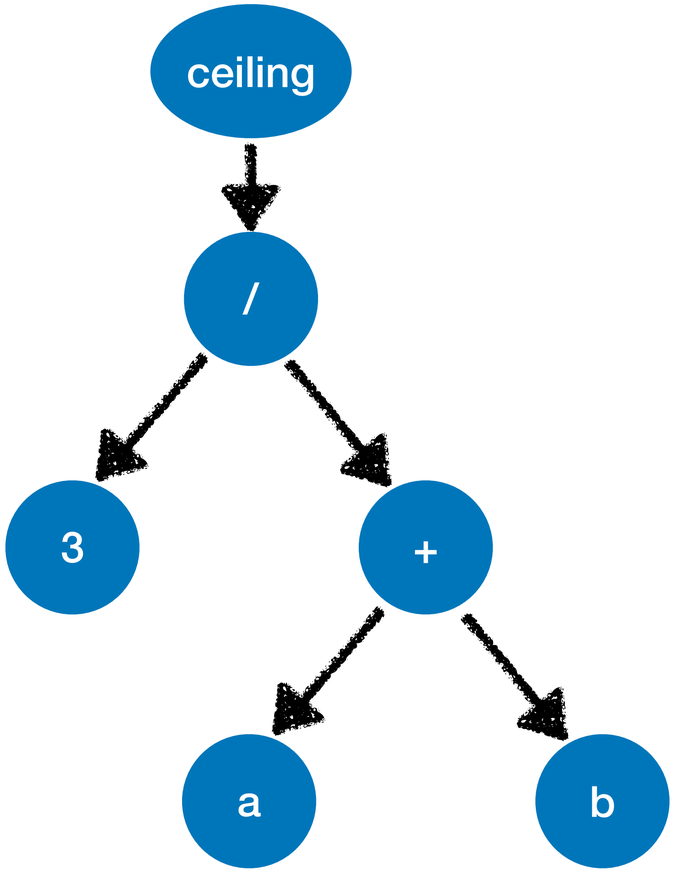
Previously: Parsers and Grammars

Parser: the cook

```
(3 / (a + b))
   . ceiling()
```



```
expression ::=
   expression binop expression
expression ::=
   expression.message(args)
…
```



Your Program: the ingredients

Grammar: the recipe

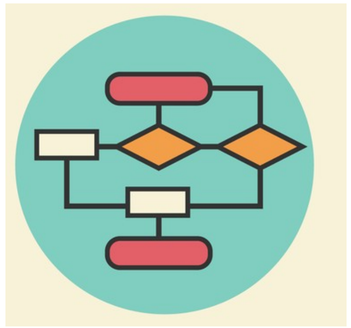
Something the machine can eat.
A data structure

Previously: Parsers and Grammars

```
(3 / (a + b))
   . ceiling()
```

Your Program: the ingredients

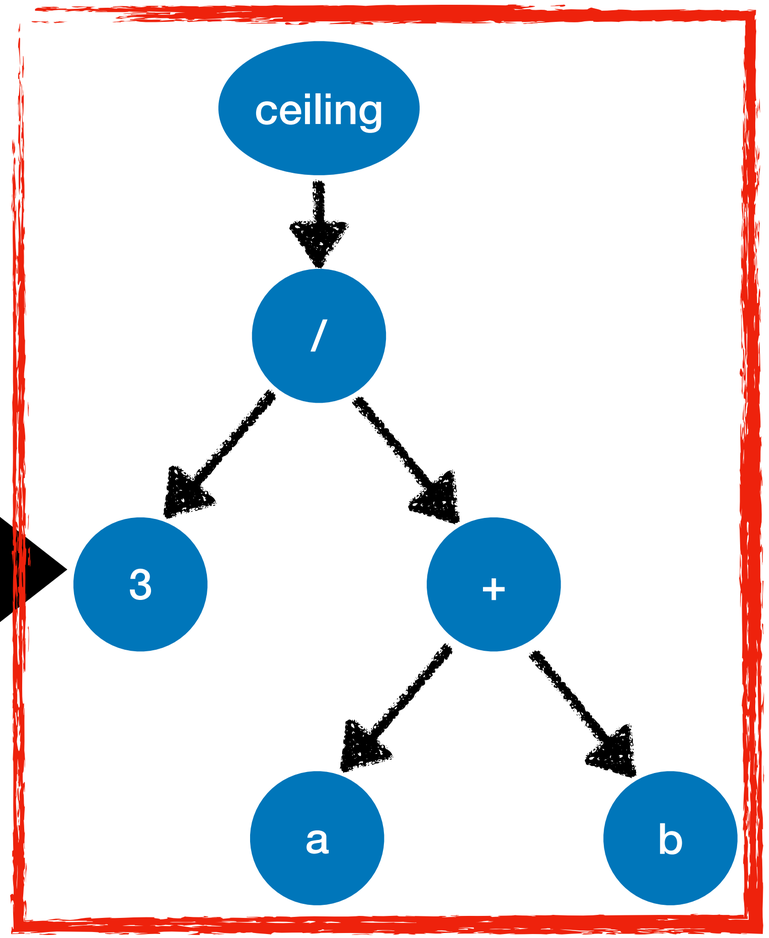
Parser: the cook



```
expression ::=
   expression binop expression
expression ::=
   expression.message(args)
…
```

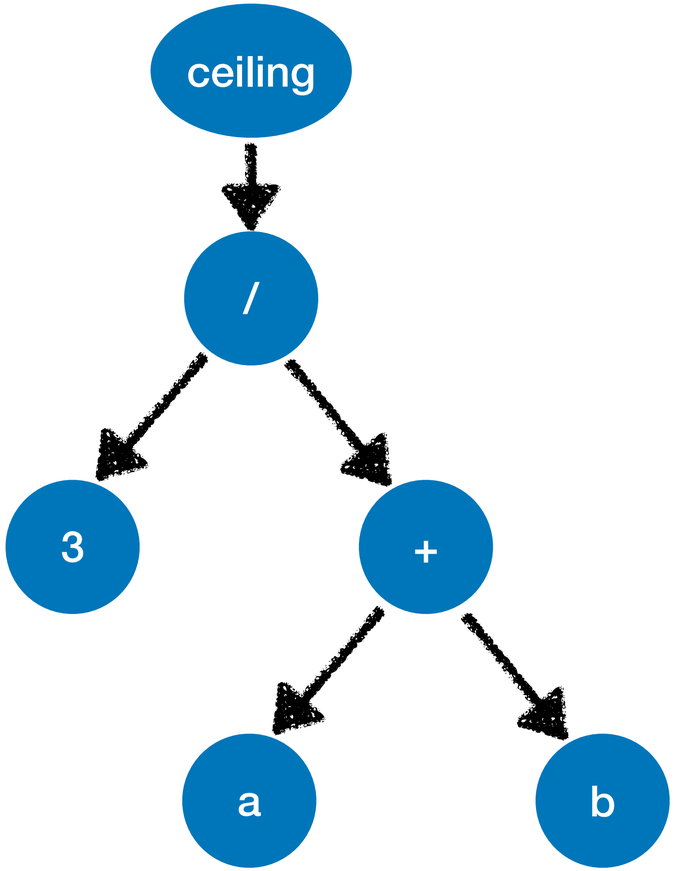
Grammar: the recipe

Parse Trees



Something the machine can eat.
A data structure

Modeling Code with Parse Trees

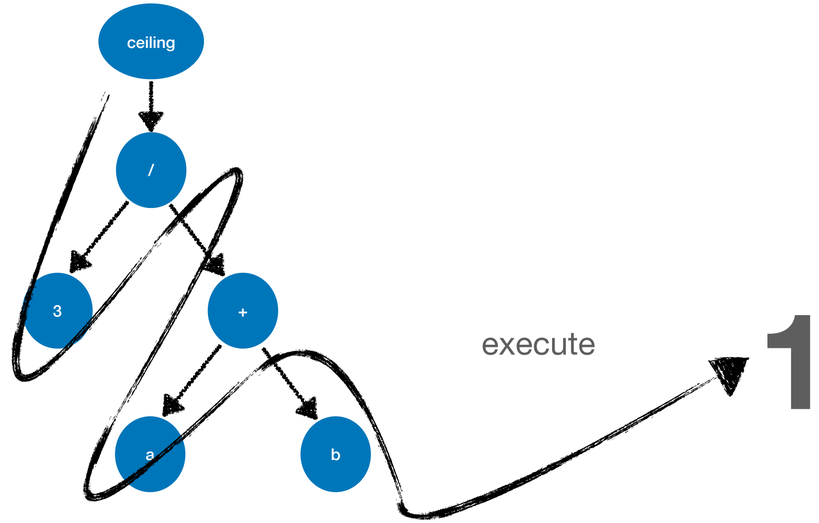


- Tree data structures that model code
- Typically the result of a parsing stage

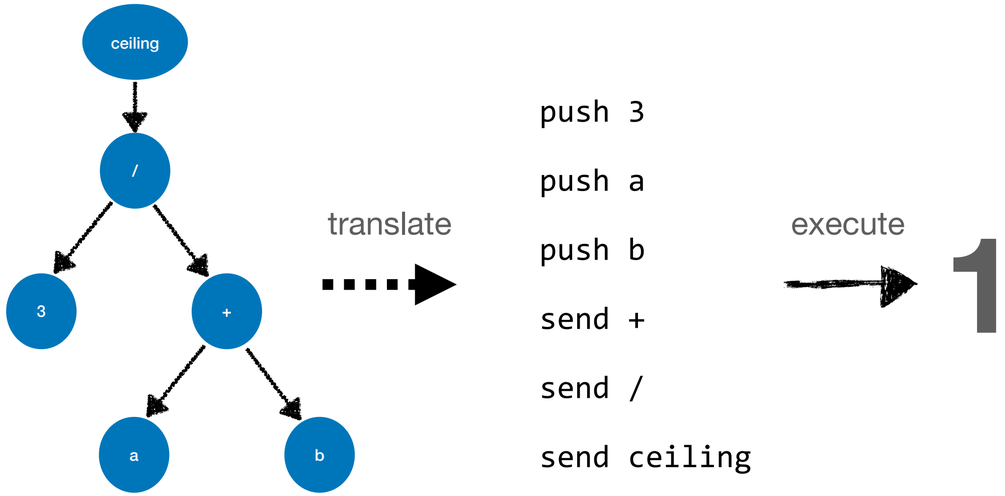
- **Concrete Syntax Trees:** contain source code information
  - *e.g.,* token position in the text
- **Abstract Syntax Trees:** just contain the minimal information for compilation
- Implementations *vary*, and may be *hybrid*

Executing Parse Trees

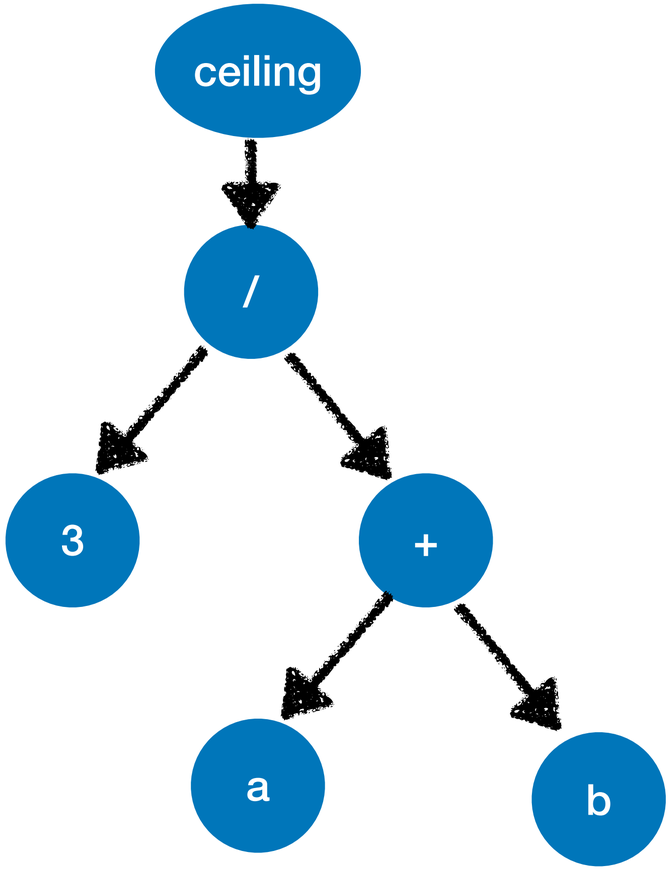
- Build a **tree-walking interpreter**
  - **Simple** to implement and understand
  - **Slow** to execute



- **Translate it** to (virtual?) machine code
  - The core topic of this course
  - Native (virtual) execution
  - Lots of optimization opportunities



Execution Overview




```
push 3

push a

push b

send +

send /

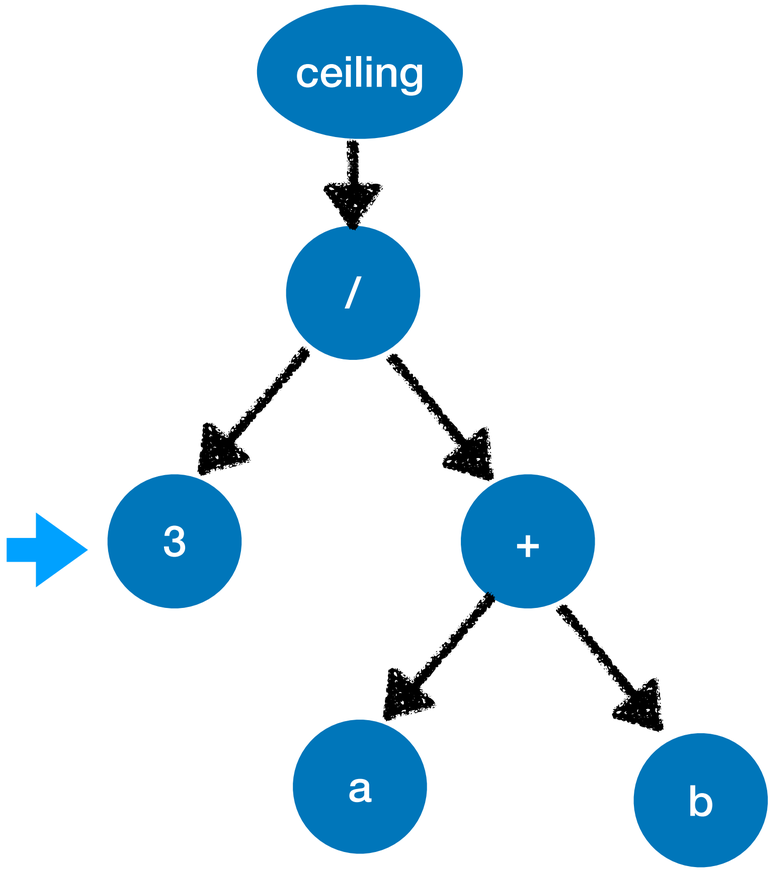
send ceiling
```

```
(3 / (a + b)) . ceiling()
```



the stack

Execution Overview



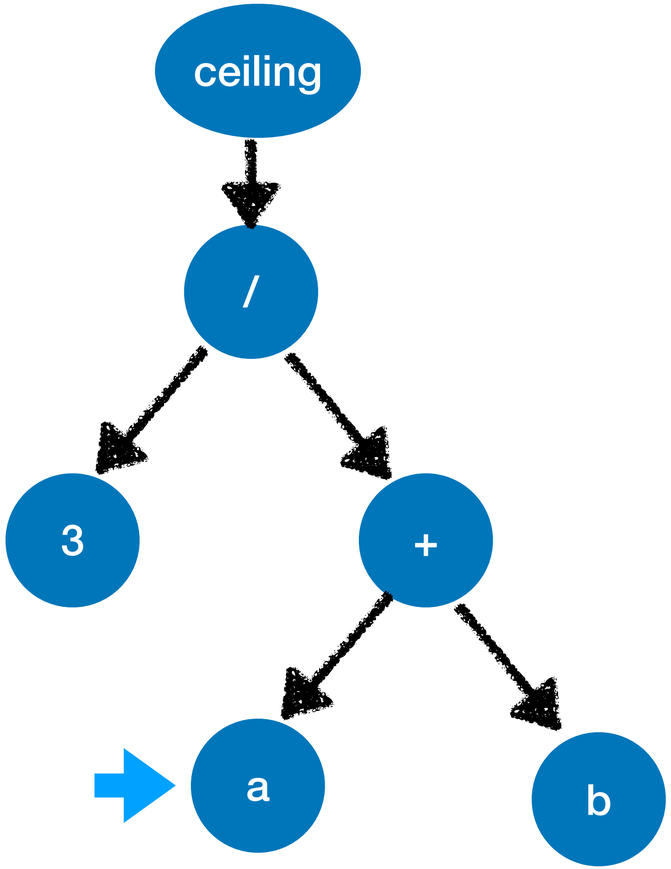

```
(3 / (a + b)) . ceiling()
```

```
push 3
push a
push b
send +
send /
send ceiling
```



the stack

Execution Overview



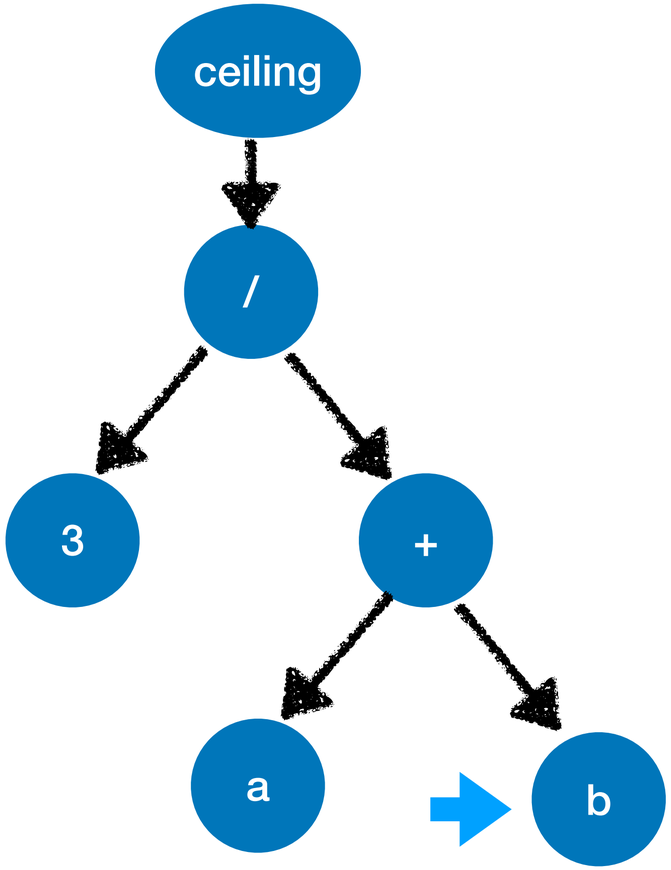

```
(3 / (a + b)) . ceiling()
```

```
push 3
push a
push b
send +
send /
send ceiling
```



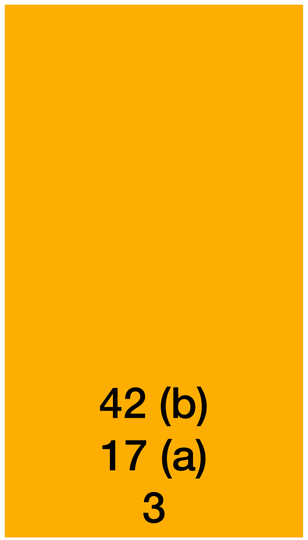
the stack

Execution Overview



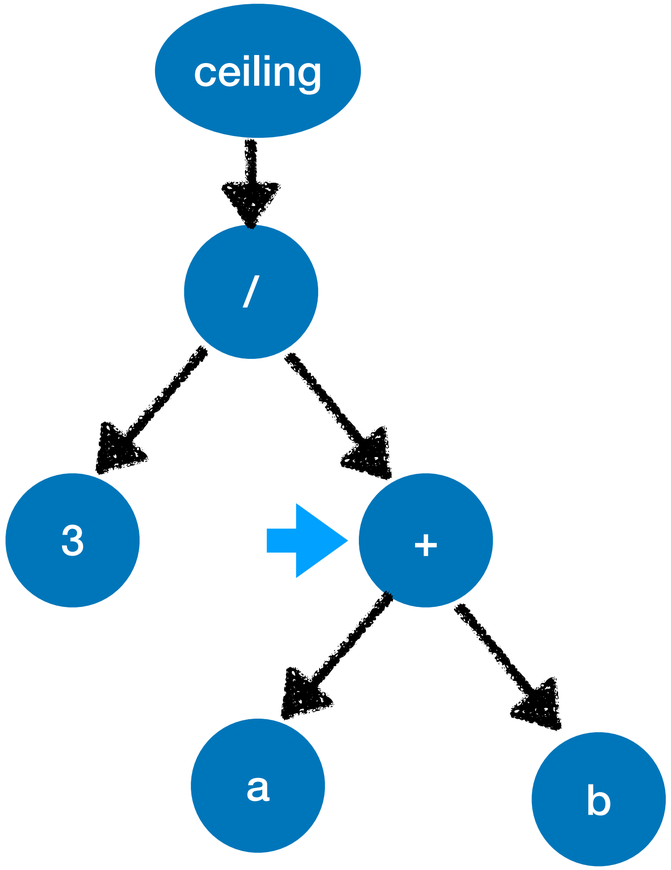

```
push 3
push a
push b
send +
send /
send ceiling
```

```
(3 / (a + b)) . ceiling()
```



the stack

Execution Overview



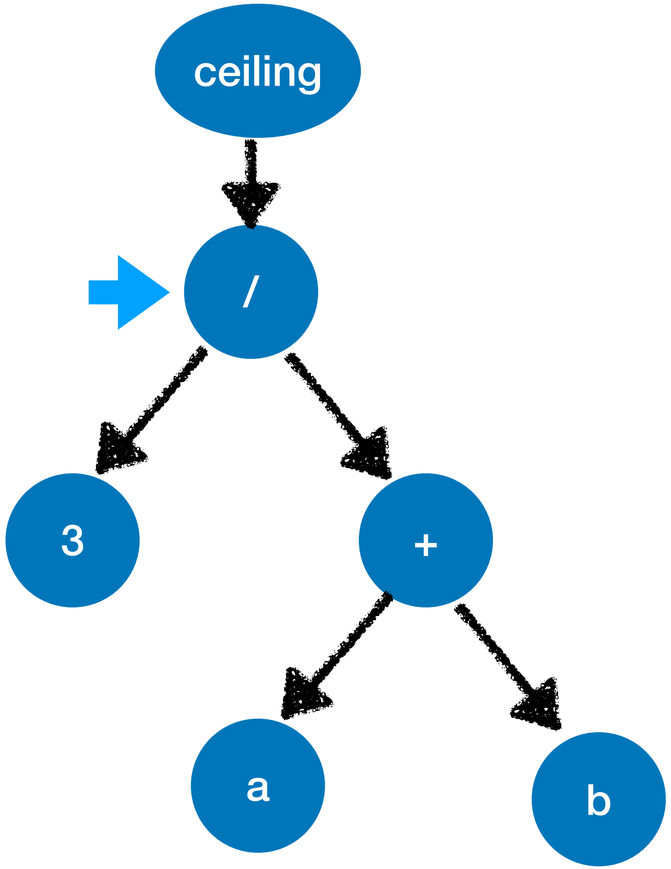

```
(3 / (a + b)) . ceiling()
```

```
push 3
push a
push b
send +
send /
send ceiling
```



the stack

Execution Overview



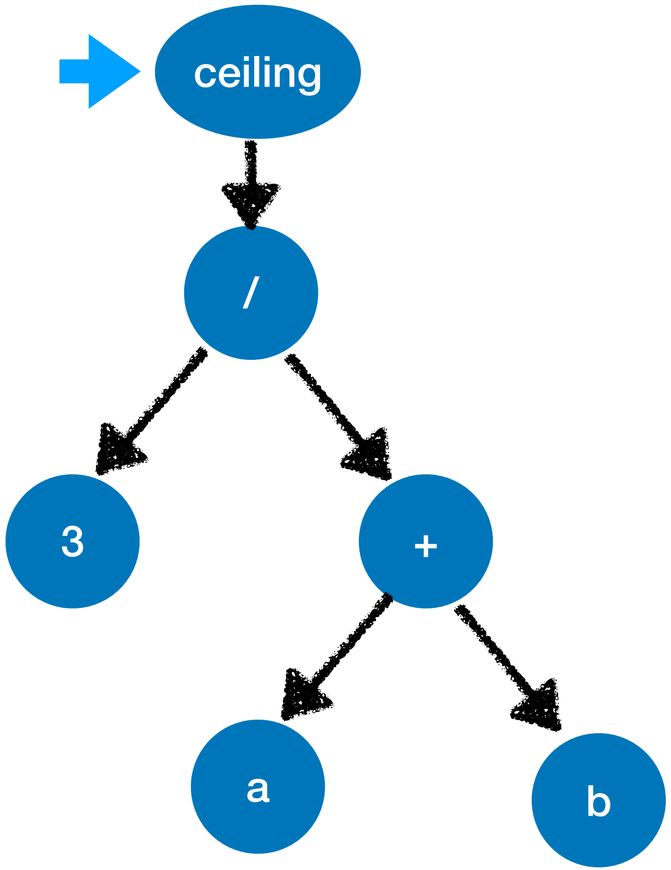

```
(3 / (a + b)) . ceiling()
```

```
push 3
push a
push b
send +
send /
send ceiling
```



the stack

Execution Overview




```
push 3
push a
push b
send +
send /
send ceiling
```

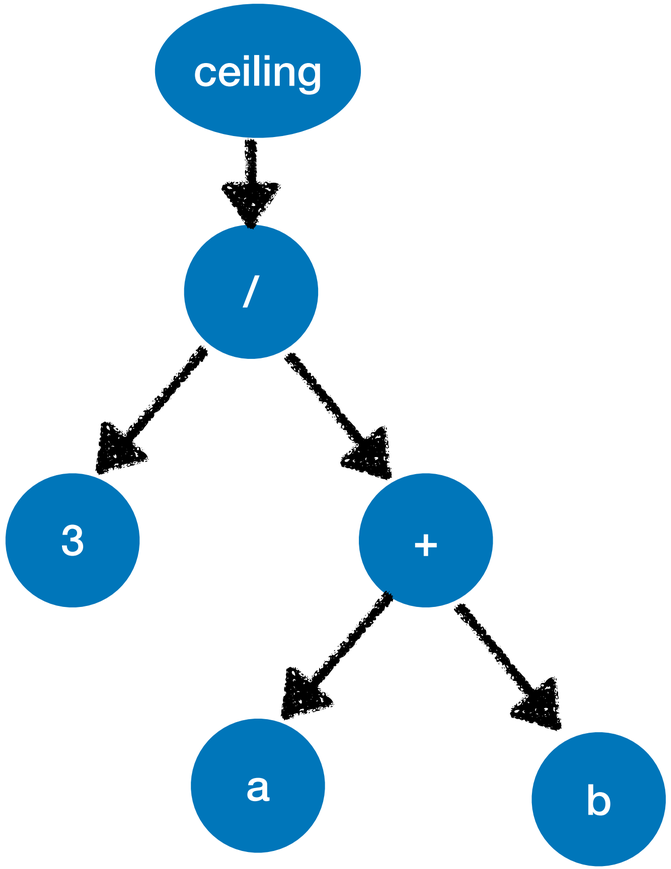
```
(3 / (a + b)) . ceiling()
```



the stack

Goal: Translate Tree to Stack Code

```
(3 / (a + b)) . ceiling()
```



bytecode compiler

```
push 3
push a
push b
send +
send /
send ceiling
```

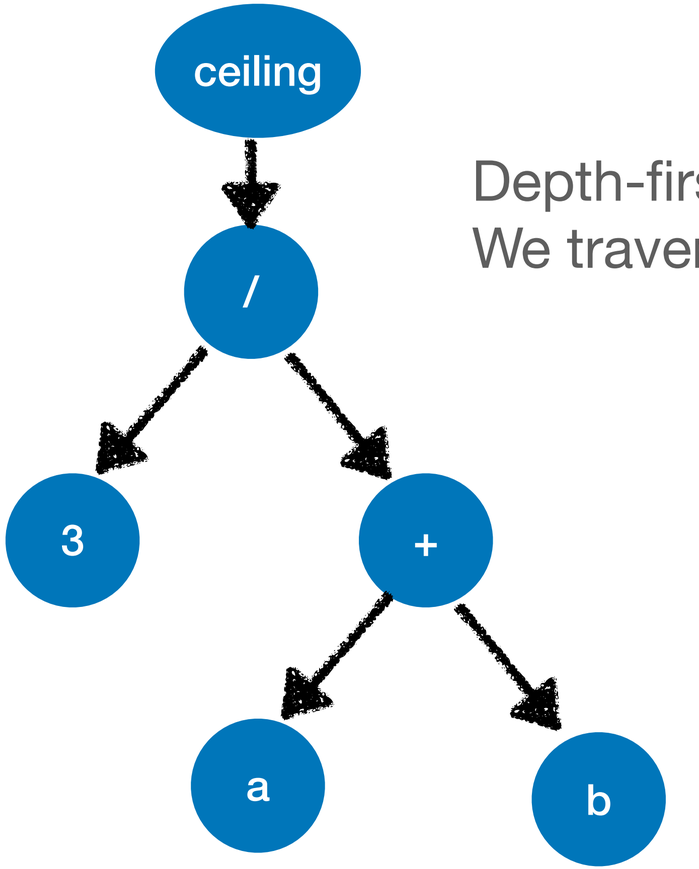
Translation Insights

- We should **traverse the tree** in the same order as we would executed it
  - *e.g.,* **post-order** depth-first, left to right (if the language semantics say so)

- **Generate** the instruction **for each node** when visiting it
  - *i.e.,* after visiting its children — post-order!
  - *Order is important*: to evaluate an instruction, operands must be available

Translation Overview

```
(3 / (a + b)) . ceiling()
```



Depth-first post-order.
We traverse the tree bottom up.

```
push 3

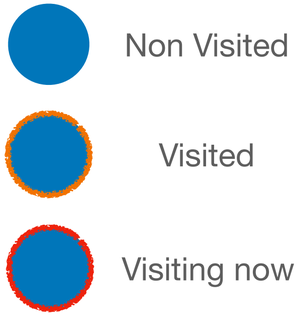
push a

push b

send +

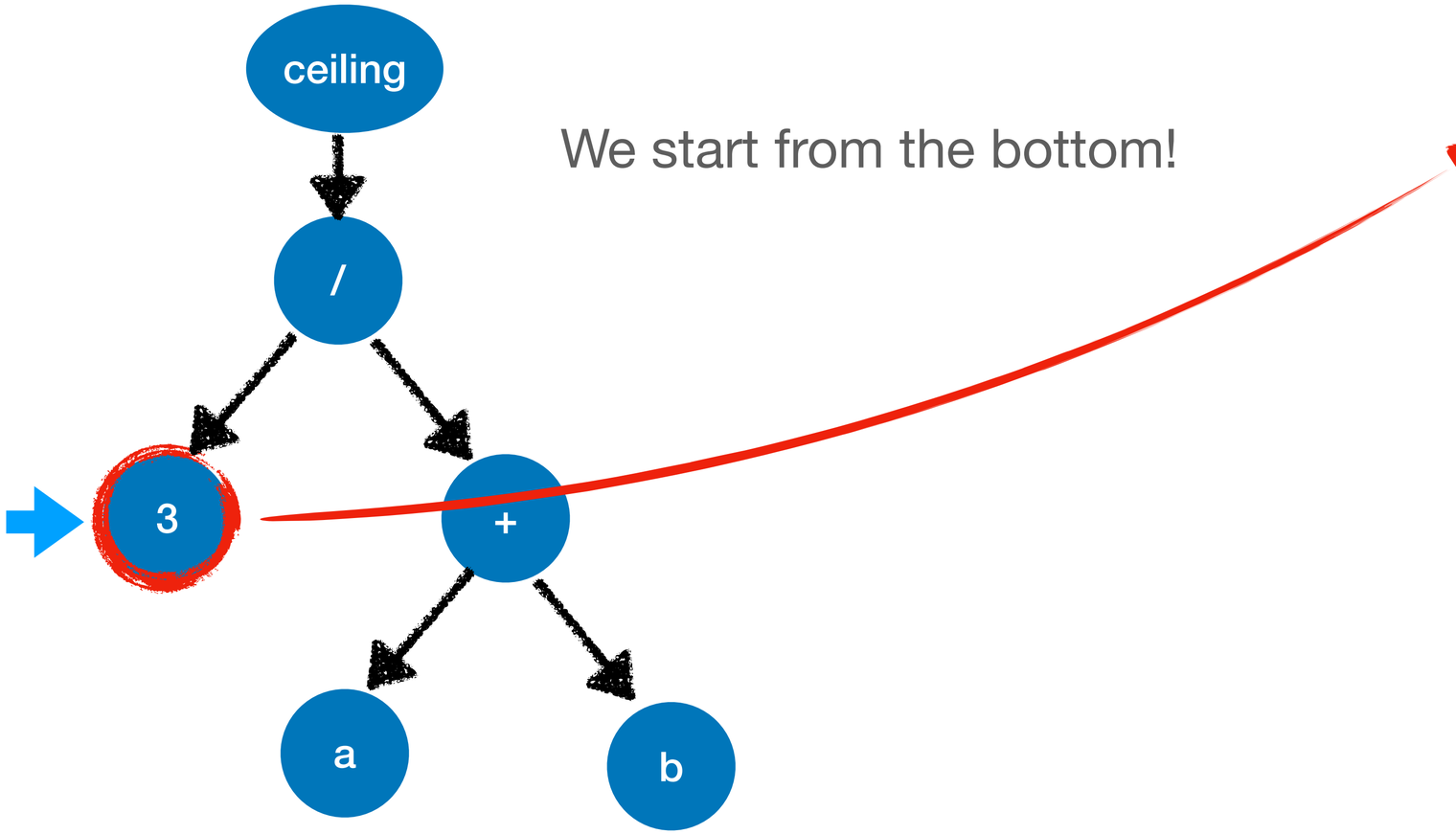
send /

send ceiling
```



Translation Overview

```
(3 / (a + b)) . ceiling()
```



We start from the bottom!

```
push 3
push a
push b
send +
send /
send ceiling
```

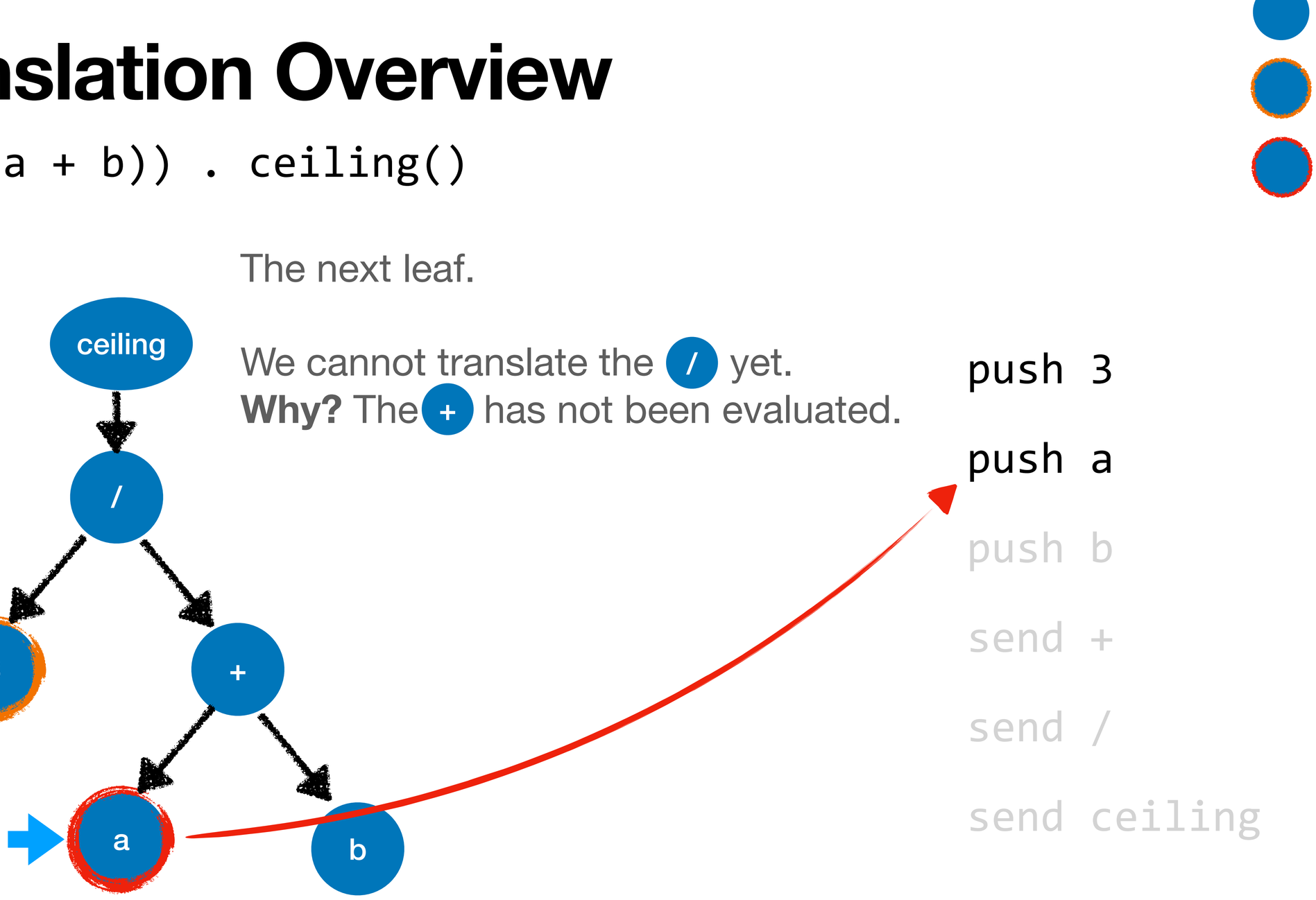
Non Visited

Visited

Visiting now

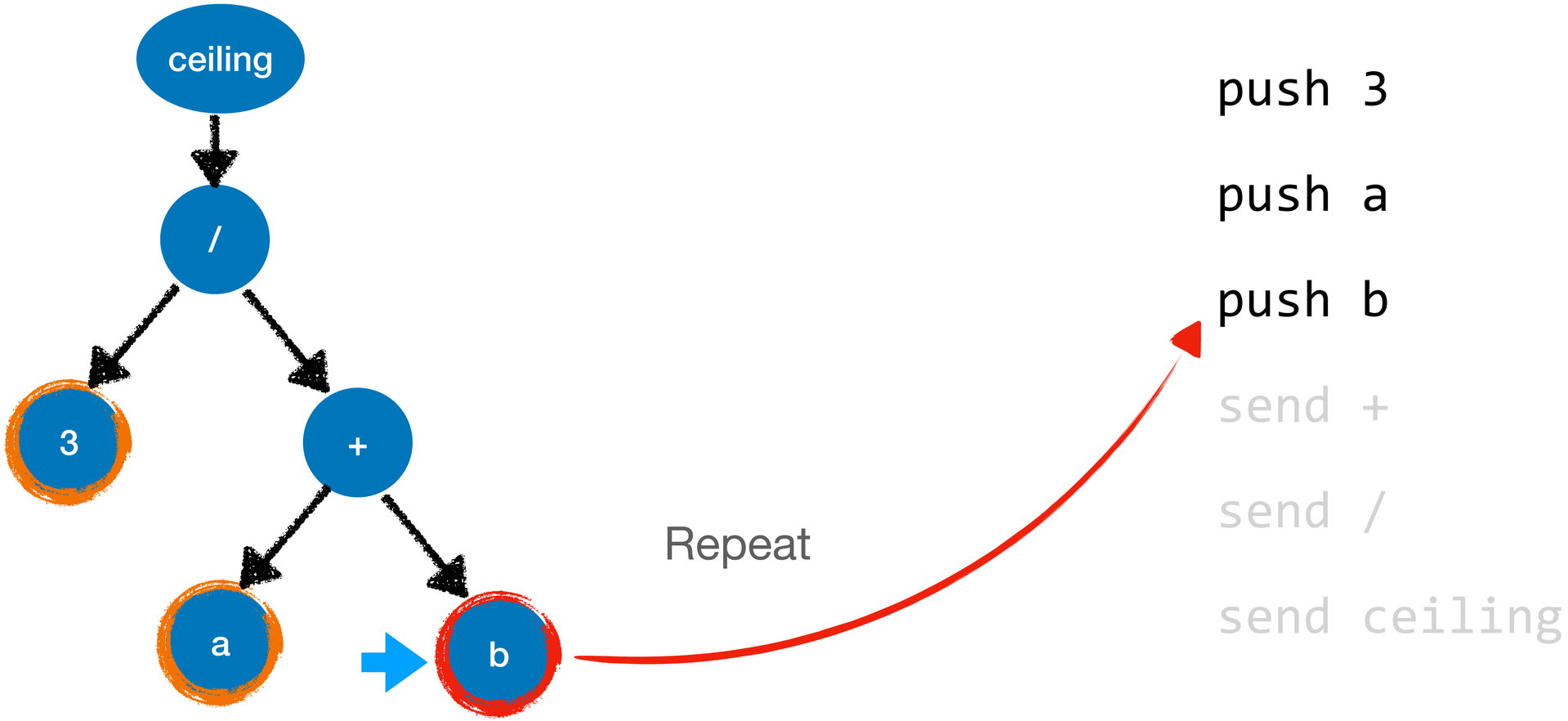
Translation Overview

```
(3 / (a + b)) . ceiling()
```



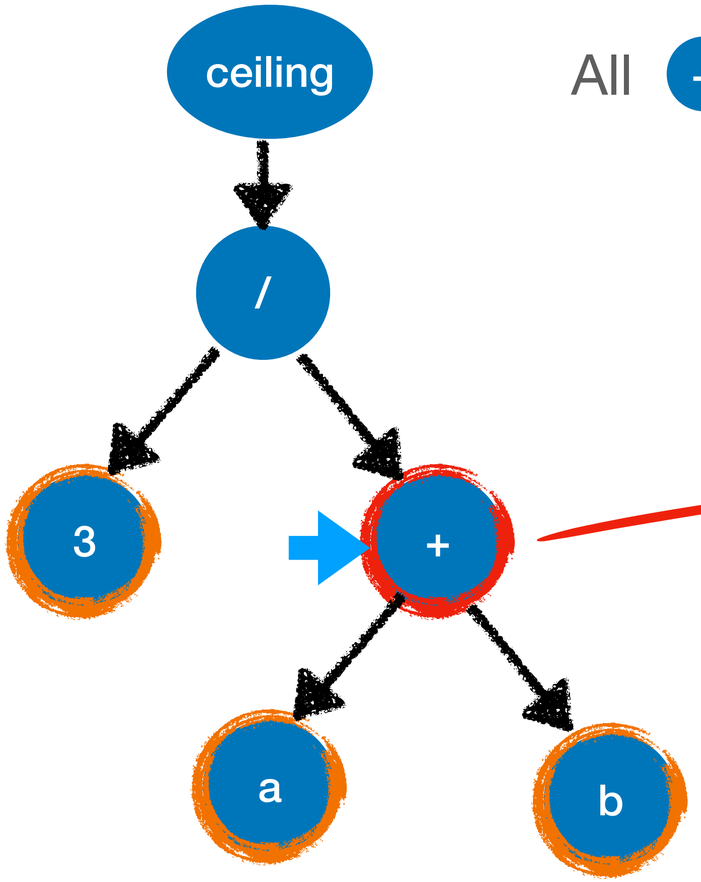
Translation Overview

```
(3 / (a + b)) . ceiling()
```



Translation Overview

```
(3 / (a + b)) . ceiling()
```



All + operands are ready!

```
push 3
push a
push b
send +
send /
send ceiling
```

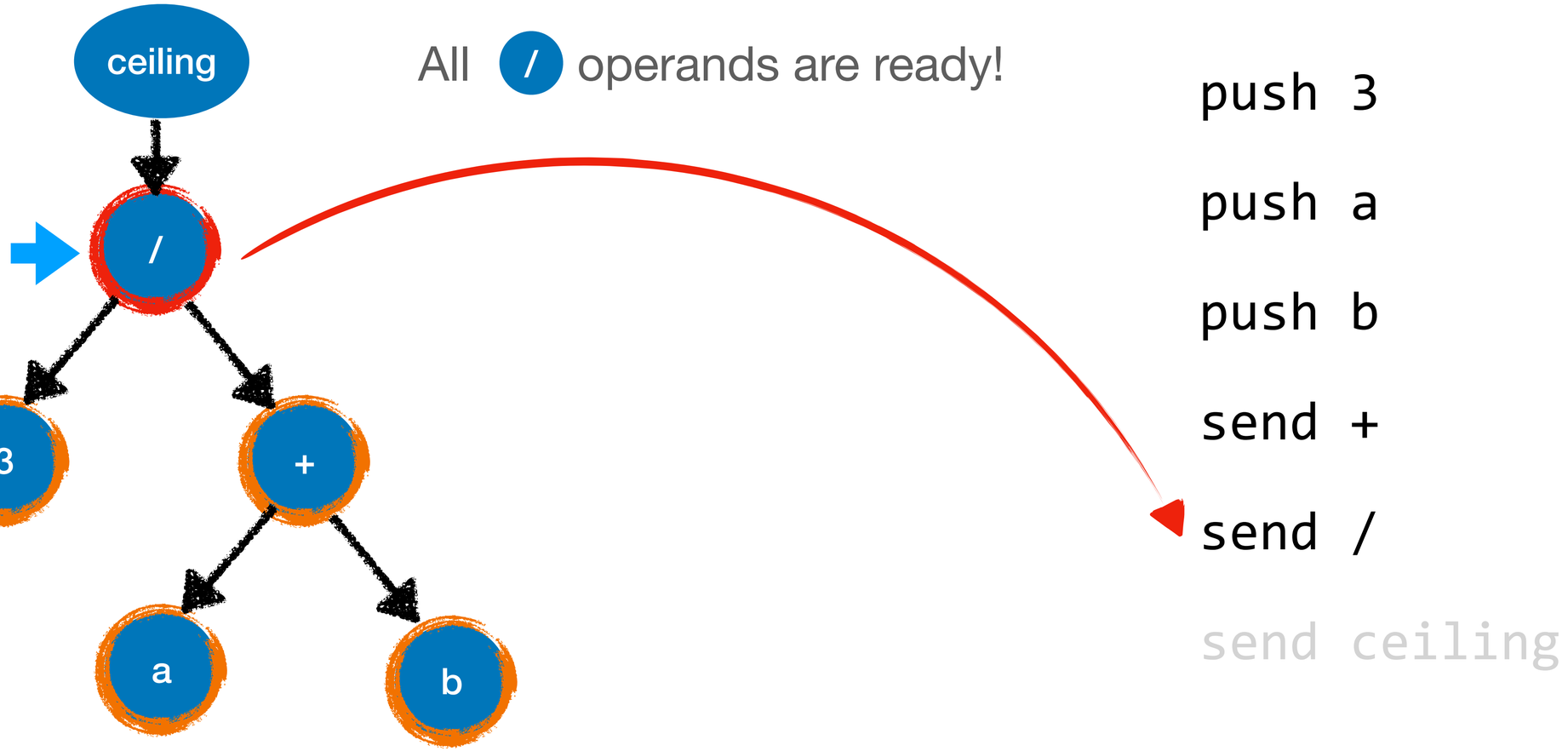
Non Visited

Visited

Visiting now

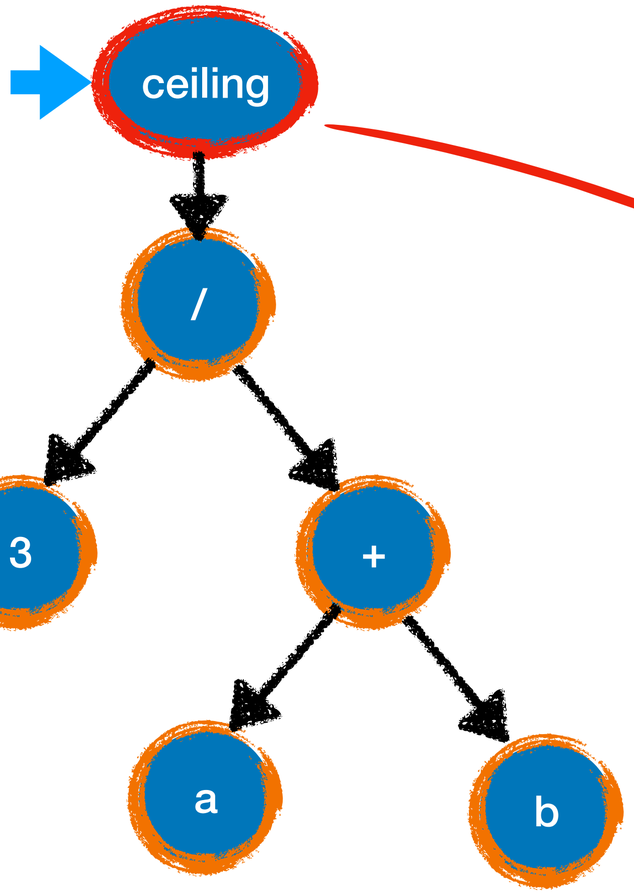
Translation Overview

```
(3 / (a + b)) . ceiling()
```



Translation Overview

```
(3 / (a + b)) . ceiling()
```




```
push 3

push a

push b

send +

send /

send ceiling
```

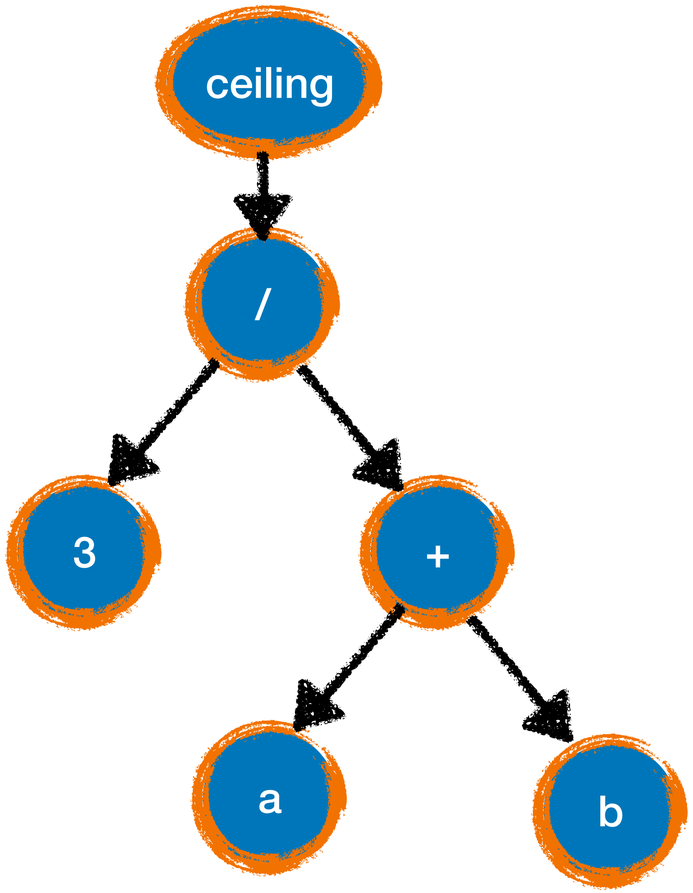
Non Visited

Visited

Visiting now

Translation Overview

```
(3 / (a + b)) . ceiling()
```



When the root is visited,
we **finished**!

```
push 3

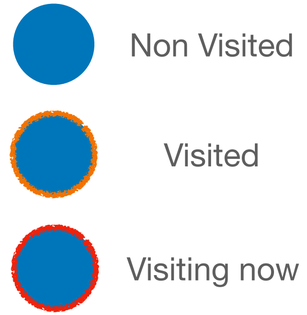
push a

push b

send +

send /

send ceiling
```



A Simple Compiler

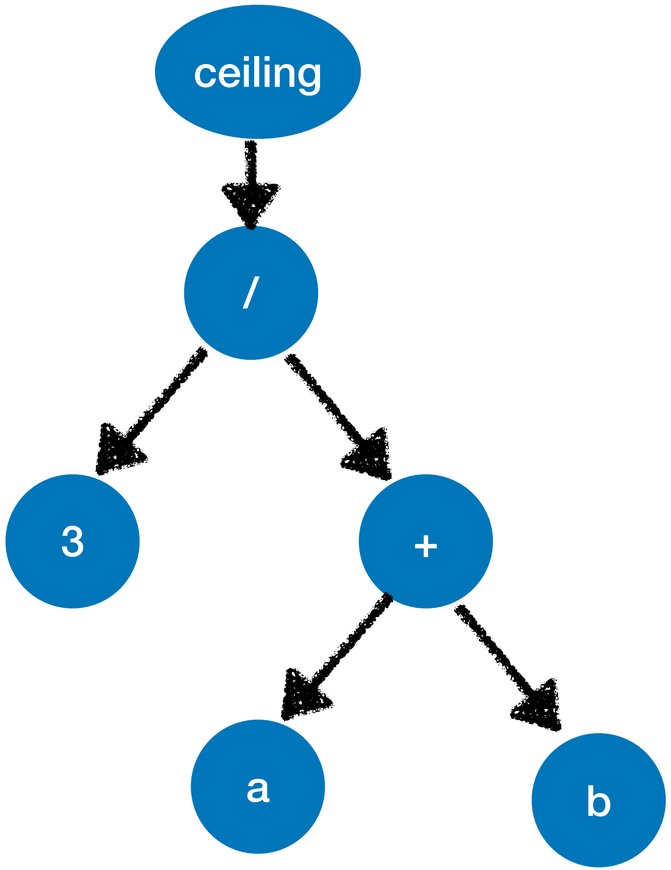
Generating code directly

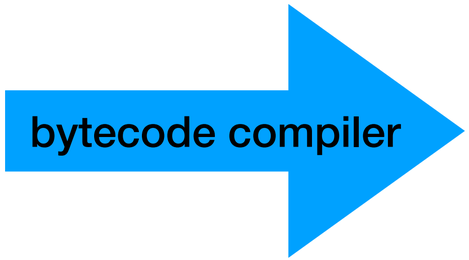
- Case analysis per type of node: *switch or visitor pattern*
- Each node encodes one or more instructions objects
- E.g.,

```
Compiler >> visitLiteralNode: aLiteralNode

        self generatePushLiteral: aLiteralNodeValue
```

Generating Binary Code Directly





17

32

33

55

56

48

opcodes

Conclusion

- Typically we will start the translation process from a *parse tree*
- Generating stack code requires same tree traversal as interpreting it

- The translation can generate directly binary code, or some another intermediate data structure or textual representation (e.g., assembly code)

Material

- Crafting interpreters book

- Tree traversals from other lectures
- Read about visitor and interpreter pattern GOF